## 🔹 1. Data Types & Variables

### 🔍 Summary:

Python has dynamic typing, but its variables still reference specific **built-in data types**, including:

| Type | Example |
| --- | --- |
| `int` | `x = 10` |
| `float` | `y = 3.14` |
| `str` | `s = "hello"` |
| `bool` | `flag = True` |
| `list` | `[1, 2, 3]` |
| `tuple` | `(1, 2)` |
| `dict` | `{"a": 1}` |
| `set` | `{1, 2, 3}` |

Python variables are just **references** to objects in memory.

### ✅ Example:

```
x = [1, 2, 3]
y = x
y.append(4)
print(x)  # [1, 2, 3, 4] — because both reference the same object
```

### 💬 Interview Q&A:

**Q1: Is Python statically or dynamically typed? A1:** Dynamically typed — variable types are determined at runtime.

**Q2: Are Python variables passed by value or reference? A2:** Technically, **by object reference**. Mutables can be changed in-place, immutables cannot.

---

**Q3: What is the difference between `is` and `==`? A3:**

- `==` checks value equality
- `is` checks object identity

---

**Q4: What's the difference between list and tuple? A4:** Lists are **mutable**, tuples are **immutable** — tuples are also faster and hashable.

---

## 🔹 2. Control Flow

---

### 🔍 Summary:

Control flow structures define how code executes based on conditions or loops.

---

### ✅ Common Control Structures:

```
# if-else
if x > 0:
    print("positive")
elif x < 0:
    print("negative")
else:
    print("zero")

# for loop
for i in range(5):
    print(i)

# while loop
while x < 10:
    x += 1

# break, continue
```

### 💬 Interview Q&A:

**Q1: When would you use `for-else`? A1:** The `else` runs only if the loop completes **without** `break`.

```
for i in range(5):
    if i == 3:
        break
else:
    print("Completed")  # Won't print
```

---

**Q2: What's the difference between `continue` and `pass`? A2:**

- `continue`: skips to next iteration
- `pass`: does nothing, placeholder for empty blocks

---

**Q3: Can Python have switch/case? A3:** Not natively, but `match-case` was added in Python 3.10.

---

## 🔹 3. Functions

---

### 🔍 Summary:

Python functions are **first-class objects** — they can be passed around, assigned to variables, or nested.

---

### ✅ Example:

```
def greet(name="Guest"):
    return f"Hello, {name}!"

say = greet
print(say("John"))  # Hello, John
```

---

### 💬 Interview Q&A:

**Q1: What are *args and **kwargs used for? A1:

- `*args` : captures **positional** arguments
- `**kwargs` : captures **keyword** arguments

```
def func(*args, **kwargs): ...
```

---

**Q2: What is a closure in Python? A2:** A function that **remembers variables from its enclosing scope**, even after that scope has finished executing.

```
def outer():
    x = 10
    def inner():
        return x
    return inner
```

---

**Q3: What is the use of `nonlocal` ? A3:** To modify a variable from the **outer non-global scope** in nested functions.

---

**Q4: Are functions objects in Python? A4:** ✅ Yes — they are instances of `function` and can be passed, stored, or returned.

---

## 🔹 4. OOP (Object-Oriented Programming)

---

### 🔍 Summary:

Python supports class-based OOP, including **inheritance**, **encapsulation**, **polymorphism**, and **abstraction**.

---

### ✅ Class Example:

```python
class Animal:
    def speak(self):
        return "Sound"

class Dog(Animal):
    def speak(self):
        return "Bark"

a = Dog()
print(a.speak())  # Bark
```

---

## 💬 Interview Q&A:

**Q1: What are `__init__` and `self`? A1:**

- `__init__` is the constructor
- `self` refers to the current instance (like `this` in Java)

---

**Q2: Does Python support multiple inheritance? A2:** ✅ Yes — Python allows it and resolves conflicts using **MRO (Method Resolution Order)**.

---

**Q3: What are dunder (magic) methods? A3:** Special methods like `__str__`, `__len__`, `__eq__` used to **override default behavior**.

---

**Q4: Difference between classmethod and staticmethod? A4:**

```python
@classmethod
def cls_method(cls): ...

@staticmethod
def static(): ...
```

- `classmethod`: receives `cls`, can access class vars
- `staticmethod`: receives nothing automatically

---

## 🔹 5. Exception Handling

---

### 🔍 Summary:

Python uses `try-except` blocks to **gracefully handle errors** without crashing the program.

---

### ✅ Example:

```
try:
    result = 10 / 0
except ZeroDivisionError:
    result = 0
finally:
    print("Done")
```

---

### 💬 Interview Q&A:

**Q1: What's the difference between `except Exception` and `except BaseException`? A1:**

- `Exception`: catches most runtime errors
- `BaseException`: also catches `KeyboardInterrupt`, `SystemExit` — generally **not recommended**

---

**Q2: Can you catch multiple exceptions? A2:** ✅ Yes:

```
except (ValueError, TypeError) as e:
    ...
```

---

**Q3: What's the use of `else` in try-except? A3:** Executes only if no exception is raised:

```
try:
    x = 5
except:
    ...
```

```
else:
    print("Success!")
```

---

**Q4: Should we always use bare `except:`? A4:** ❌ No — it catches everything, including system-exiting exceptions.

---

## 🔹 6. File Handling

---

### 🔍 Summary:

Python handles files using the built-in `open()` method and supports reading, writing, and appending.

---

### ✅ Example:

```
with open("data.txt", "r") as f:
    contents = f.read()
```

---

### 💬 Interview Q&A:

**Q1: Why use `with open()`? A1:** It ensures **automatic resource management** — the file is closed even if exceptions occur.

---

**Q2: What modes can you use with `open()`? A2:**

- `'r'`: read
- `'w'`: write (overwrite)
- `'a'`: append
- `'rb'`, `'wb'`: binary modes

---

**Q3: How to read files line by line? A3:**

```
for line in open("file.txt"):
    print(line)
```

---

**Q4: How to write and append to the same file? A4:** Open with `'a+'` mode — appends and allows read from beginning.

---

## 🔹 7. Modules & Packages

---

### 🔍 Summary:

A **module** is a `.py` file. A **package** is a directory with `__init__.py`. Python uses the `import` system to structure and reuse code.

---

### ✅ Example:

```
# math_utils.py
def add(a, b):
    return a + b

# main.py
from math_utils import add
```

---

### 💬 Interview Q&A:

**Q1: What's the difference between module and package? A1:**

- **Module** = single file
- **Package** = directory with `__init__.py`, can contain multiple modules

---

**Q2: What is `__all__` used for? A2:** It controls what gets imported when using:

```
from mypkg import *
```

**Q3: How do you make a module executable as a script? A3:**

```
if __name__ == "__main__":
    main()
```

---

**Q4: How does Python locate modules? A4:** Via `sys.path`, which includes current dir, site-packages, and PYTHONPATH.

---

## 🔹 8. List Comprehensions

---

### 🔍 Summary:

List comprehensions offer a **compact syntax** for creating lists from iterables.

---

### ✅ Example:

```
squares = [x*x for x in range(5)]
```

Also supports conditions:

```
evens = [x for x in range(10) if x % 2 == 0]
```

---

### 💬 Interview Q&A:

**Q1: How is list comprehension different from `map()`? A1:** List comprehensions are **more readable** and allow **filtering inline**, while `map()` is more functional-style.

---

**Q2: Can list comprehensions be nested? A2:** ✅ Yes:

```
matrix = [[i*j for j in range(3)] for i in range(3)]
```

---

**Q3: Can you use `if-else` inside list comprehension? A3:** ✅ Yes:

```
["even" if x%2==0 else "odd" for x in range(5)]
```

---

**Q4: When should you avoid list comprehensions? A4:** For very complex logic — use regular loops for clarity.

---

Great — now we're stepping into some of Python's **most powerful internals**: functions that remember state, decorators, iteration protocols, and how variables actually resolve. These topics are **frequent in intermediate–advanced interviews**.

---

## ✅ Topics:

**9. Decorators 10. Generators 11. Iterators 12. Namespaces & Scope**

---

# 🔹 9. Decorators

---

## 🔍 Summary:

A **decorator** is a function that **wraps another function**, modifying or enhancing its behavior without changing its code.

---

## ✅ Example:

```
def log_decorator(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper
```

```
@log_decorator
def greet(name):
    return f"Hello, {name}"
```

---

## 💬 Interview Q&A:

**Q1: What are decorators commonly used for? A1:**

- Logging
- Authentication
- Caching
- Access control
- Timing and performance monitoring

---

**Q2: What's the difference between `@staticmethod`, `@classmethod`, and custom decorators? A2:**

- `@staticmethod`: no `self` or `cls`
- `@classmethod`: receives `cls`
- Custom: defined using closures or `functools.wraps`

---

**Q3: How do you preserve function metadata in decorators? A3:**

```
from functools import wraps

@wraps(func)
def wrapper(...):
```

---

**Q4: Can decorators be stacked? A4:** ✅ Yes — they are applied bottom-up:

```
@decorator1
@decorator2
def func(): ...
```

---

## 🔹 10. Generators

### 🔍 Summary:

Generators **yield values one at a time**, maintaining state between calls. They use **less memory** than returning lists.

### ✅ Example:

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1
```

### 💬 Interview Q&A:

**Q1: How are generators different from normal functions? A1:** Generators use `yield` instead of `return` and produce a **lazy iterator**.

**Q2: Why use generators? A2:** For memory-efficient iteration over large or infinite sequences.

**Q3: How do you manually iterate a generator? A3:**

```
gen = countdown(3)
next(gen)
```

**Q4: Can generators be recursive? A4:** ✅ Yes, but use `yield from` to yield from sub-generators.

## 🔹 11. Iterators

### 🔍 Summary:

An **iterator** is any object that implements the `__iter__()` and `__next__()` methods.

---

### ✅ Example:

```
class Count:
    def __init__(self, n):
        self.n = n
    def __iter__(self):
        return self
    def __next__(self):
        if self.n <= 0:
            raise StopIteration
        self.n -= 1
        return self.n
```

---

### 💬 Interview Q&A:

**Q1: What's the difference between iterable and iterator? A1:**

- **Iterable**: has `__iter__()`
- **Iterator**: has both `__iter__()` and `__next__()`

---

**Q2: Is `list` an iterator? A2:** ❌ No — it's an **iterable**. You must call `iter(list)` to get an iterator.

---

**Q3: What happens when an iterator is exhausted? A3:** It raises a `StopIteration` exception.

---

**Q4: Are generators also iterators? A4:** ✅ Yes — they implement both `__iter__()` and `__next__()`.

---

## 🔹 12. Namespaces & Scope

---

### 🔍 Summary:

Python resolves variable names using the **LEGB rule**:

| Scope | Meaning |
|---|---|
| L | Local (inside function) |
| E | Enclosing (nonlocal scope) |
| G | Global (module-level) |
| B | Built-in ( `len` , `sum` ) |

---

## ✅ Example:

```
x = 5  # Global

def outer():
    x = 10  # Enclosing
    def inner():
        nonlocal x
        x += 1
    inner()
    return x
```

---

## 💬 Interview Q&A:

**Q1: What does `nonlocal` do? A1:** Allows you to modify a variable from an enclosing (non-global) scope.

---

**Q2: Can you assign to a global variable inside a function? A2:** ✅ Yes — using `global` :

```
global x
x = 100
```

---

**Q3: What's the difference between `locals()` and `globals()` ? A3:**

- `locals()` : current local scope as a dict
- `globals()` : module/global scope as a dict

---

**Q4: What is shadowing in Python? A4:** When a local variable overrides a global or built-in name (e.g., defining `sum = 10`).

---

## 🔹 13. Memory Management

### 🔍 Summary:

Python manages memory via:

- **Reference counting**
- **Garbage collection (GC)**
- **Heap allocation for objects**
- **Memory pools (via `pymalloc`)**

---

### ✅ GC Trigger Example:

```
import gc

gc.collect()  # manually triggers garbage collection
```

---

### 💬 Interview Q&A:

**Q1: How does Python detect memory that can be freed? A1:** Through **reference counting** and cycle detection via the **garbage collector**.

---

**Q2: What is a memory leak in Python? A2:** When objects stay in memory due to lingering references (e.g., closures, circular references).

---

**Q3: What module can track memory usage? A3:** `gc`, `tracemalloc`, or third-party tools like `memory_profiler`, `objgraph`.

---

**Q4: Why is CPython's memory model slower than Java? A4:** Python prioritizes flexibility over raw speed; uses dynamic typing, late binding, and interpreter overhead.

---

## 🔹 14. RegEx (Regular Expressions)

---

### 🔍 Summary:

Python uses the `re` module for pattern matching, substitution, and splitting based on text patterns.

---

### ✅ Example:

```
import re

pattern = r"\b\d{3}-\d{2}-\d{4}\b"
text = "My SSN is 123-45-6789"
match = re.search(pattern, text)
```

---

### 💬 Interview Q&A:

**Q1: Difference between `search()` and `match()`? A1:**

- `match()`: checks only at beginning of string
- `search()`: checks anywhere in string

---

**Q2: What's the use of `findall()`? A2:** Returns a list of **all matches** in the string.

---

**Q3: What does `\b` mean in regex? A3: Word boundary —** ensures match only at edges of words.

---

**Q4: How to substitute matches? A4:**

```
re.sub(r'\d+', 'X', 'abc123')  # → 'abcX'
```

## 🔹 15. Comprehensions

---

### 🔍 Summary:

Comprehensions are one-liners for creating collections. Python supports:

- List
- Set
- Dict
- Generator comprehensions

---

### ✅ Examples:

```
# Set
{x for x in range(5)}

# Dict
{k: k**2 for k in range(3)}

# Generator (lazy)
(x**2 for x in range(5))
```

---

### 💬 Interview Q&A:

**Q1: How are set/dict comprehensions useful? A1:** They provide **concise syntax** for filtering or transforming collections.

---

**Q2: When should you use generator comprehensions? A2:** When memory efficiency is critical — generators don't store items in memory.

---

**Q3: Can you nest comprehensions? A3:** ✅ Yes, but clarity suffers:

```
[(i, j) for i in range(3) for j in range(3)]
```

---

**Q4: Difference between list comprehension and generator expression? A4:**

- `[x for x in ...]` : materializes list
- `(x for x in ...)` : yields one at a time

---

## 🔹 16. FP Tools: `map` , `filter` , `lambda`

---

### 🔍 Summary:

Python supports **functional-style programming** via:

- `map(func, iterable)`
- `filter(func, iterable)`
- `lambda` (anonymous functions)

---

### ✅ Example:

```
nums = [1, 2, 3, 4]
squared = list(map(lambda x: x**2, nums))
evens = list(filter(lambda x: x % 2 == 0, nums))
```

---

### 💬 Interview Q&A:

**Q1: When to use `map()` vs list comprehension? A1:** `map()` is shorter for applying **existing functions**. List comprehensions are clearer for inline logic + filtering.

---

**Q2: Can `lambda` have multiple statements? A2:** ❌ No — it must be a **single expression**.

---

**Q3: Can `filter()` return an empty list? A3:** ✅ Yes — if no elements pass the condition.

---

**Q4: How to replace `map()` with list comp? A4:**

```
[x**2 for x in nums]  # same as map(lambda x: x**2, nums)
```

---

## 🔹 17. Multithreading

---

### 🔍 Summary:

Python supports threads using the `threading` module, but due to the **Global Interpreter Lock (GIL)**, threads are ideal for **I/O-bound tasks**, not CPU-bound.

---

### ✅ Example:

```
import threading

def worker():
    print("Running")

t1 = threading.Thread(target=worker)
t1.start()
t1.join()
```

---

### 💬 Interview Q&A:

**Q1: What is the GIL and why does it matter? A1:** The GIL allows only **one thread to execute Python bytecode at a time**, limiting true parallelism for CPU-heavy tasks.

---

**Q2: When should you use threading vs multiprocessing? A2:**

- **Threading**: good for I/O-bound (e.g., API calls, file I/O)
- **Multiprocessing**: better for CPU-bound tasks (e.g., data crunching)

---

**Q3: How do you share data between threads? A3:** Use shared memory, thread-safe structures like `Queue`, or locks (`threading.Lock`).

## 🔹 18. Asyncio

### 🔍 Summary:

`asyncio` enables **concurrent** (not parallel) I/O using an **event loop** — best for tasks like web scraping, bots, or APIs.

### ✅ Example:

```
import asyncio

async def say_hi():
    await asyncio.sleep(1)
    print("Hello")

asyncio.run(say_hi())
```

### 💬 Interview Q&A:

**Q1: Difference between threading and asyncio? A1:**

- **Threading** = OS-level threads, blocking
- **Asyncio** = single-threaded, non-blocking via event loop

**Q2: Can you use `await` in a regular function? A2:** ❌ No — only inside `async def`.

**Q3: Can `asyncio` replace threading? A3:** For **I/O-bound** tasks, yes. For CPU-bound, use multiprocessing or run in executor.

**Q4: What's `asyncio.gather()`? A4:** Runs multiple coroutines concurrently:

```
await asyncio.gather(task1(), task2())
```

## 🔹 19. Unit Testing

---

### 🔍 Summary:

Python has built-in support for unit testing using `unittest`, and commonly also `pytest` for ease and features.

---

### ✅ Example:

```
import unittest

def add(x, y):
    return x + y

class TestMath(unittest.TestCase):
    def test_add(self):
        self.assertEqual(add(2, 3), 5)
```

Run via:

```
python -m unittest test_file.py
```

---

### 💬 Interview Q&A:

**Q1: What is the difference between `assertEqual` and `assertTrue`? A1:**

- `assertEqual(a, b)` → `a == b`
- `assertTrue(x)` → `bool(x) is True`

---

**Q2: How is `pytest` different from `unittest`? A2:**

- Less boilerplate
- Uses `assert` directly
- Fixtures for setup

- Parametrized tests

---

**Q3: What are fixtures in `pytest`? A3:** Functions that **set up resources** before a test runs and clean up after.

---

**Q4: Why is mocking used in testing? A4:** To **simulate external dependencies** (like APIs, DB) using `unittest.mock`.

---

## 🔹 20. PEP 8

---

### 🔍 Summary:

PEP 8 is the **official Python style guide**, ensuring code is readable, consistent, and community-compliant.

---

### ✅ Key Points:

| Rule | Example |
|---|---|
| Indent 4 spaces | `not tabs` |
| Max 79 characters | Line wrapping |
| Blank lines | 2 before class/def |
| Naming | `snake_case`, `CamelCase` |
| Imports | 3 groups: stdlib, 3rd-party, local |

---

### 💬 Interview Q&A:

**Q1: Why follow PEP 8? A1:** For **readability, maintainability**, and to meet team standards.

---

**Q2: What tools enforce PEP 8? A2:**

- `flake8`, `black`, `pylint`, `isort`

**Q3: How to auto-format code per PEP 8? A3:**

```
black my_code.py
```

---

**Q4: What is E501? A4:** PEP 8 lint error for **line too long** (over 79 or 88 chars).

---

# 🎁 Bonus 4 Python Topics (Interview-Focused)

---

## 🔹 1. Typing & Type Hints

---

### 🔍 Summary:

Introduced in Python 3.5+, **type hints** help with readability, IDE autocompletion, and static analysis.

---

### ✅ Example:

```
def greet(name: str) -> str:
    return f"Hello, {name}"
```

### 💬 Interview Q&A:

> **Q:** What are type hints used for? **A:** They provide optional static typing and work with tools like `mypy`, `pyright`.

---

## 🔹 2. Dataclasses

---

### 🔍 Summary:

`dataclasses` (Python 3.7+) reduce boilerplate for model-like classes (like in Django/Pydantic).

✅ **Example:**

```
from dataclasses import dataclass

@dataclass
class User:
    name: str
    age: int
```

💬 **Interview Q&A:**

> **Q:** How do dataclasses compare to namedtuples or classes? **A:** More flexible than namedtuples, less verbose than regular classes; support default values, comparison, etc.

---

## 🔹 3. Context Managers ( `with` ) & `__enter__` , `__exit__`

---

🔍 **Summary:**

Used for **resource management** (like files, DBs, locks). Behind the scenes: `__enter__` , `__exit__` .

---

✅ **Example:**

```
class Timer:
    def __enter__(self):
        import time
        self.start = time.time()
        return self

    def __exit__(self, *args):
        print("Elapsed:", time.time() - self.start)

with Timer():
    sum(range(100000))
```

💬 **Interview Q&A:**

> **Q:** When should you create a custom context manager? **A:** When working with resources (sockets, files, DBs) where **setup/cleanup** is needed.

## 🔹 4. Custom Exceptions & Exception Hierarchy

### 🔍 Summary:

Creating project-specific exceptions helps distinguish error sources and improves debugging.

### ✅ Example:

```
class InvalidUserError(Exception):
    pass

def login(user):
    if user != "admin":
        raise InvalidUserError("Not allowed")
```

### 💬 Interview Q&A:

> **Q:** Why should you create custom exceptions? **A:** For **cleaner error handling**, improved logging, and precise debugging.